

Preparation Notebook

Setup Environment

In [1]: *# DO NOT MODIFY THE CODE IN THIS CELL*

```
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

12.9/12.9 MB 67.5 MB/s eta 0:00

4.9/4.9 MB 81.8 MB/s eta 0:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

In [2]:

```
group_name = "20"
student_name = "Shanu Sharma"
student_id = "25979360"
```

In [3]: *# DO NOT MODIFY THE CODE IN THIS CELL*

```
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

```
In [4]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Shanu Sharma

```
In [5]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25979360

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here
using the command: `! pip install <package_name>`

```
In [6]: #No additional packages required.
```

0.b Import Packages

```
In [7]: # DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import numpy as np
```

A. Feature Selection

A.0 Load Data

```
In [8]: # DO NOT MODIFY THE CODE IN THIS CELL
# Load datasets
try:
    customer_df = pd.read_csv(at.folder_path / "customer.csv")
    person_df = pd.read_csv(at.folder_path / "person.csv")
    product_category_df = pd.read_csv(at.folder_path / "product_category.csv")
    product_cost_history_df = pd.read_csv(at.folder_path / "product_cost_history.csv")
    product_list_price_history_df = pd.read_csv(at.folder_path / "product_list_price_history.csv")
    product_sub_category_df = pd.read_csv(at.folder_path / "product_sub_category.csv")
    product_df = pd.read_csv(at.folder_path / "product.csv")
    sales_order_detail_df = pd.read_csv(at.folder_path / "sales_order_detail.csv")
```

```

sales_order_header_df = pd.read_csv(at.folder_path / "sales_order_header.csv")
sales_territory_df = pd.read_csv(at.folder_path / "sales_territory.csv")
special_offer_product_df = pd.read_csv(at.folder_path / "special_offer_product.csv")
special_offer_df = pd.read_csv(at.folder_path / "special_offer.csv")
store_df = pd.read_csv(at.folder_path / "store.csv")
unit_measure_df = pd.read_csv(at.folder_path / "unit_measure.csv")
except Exception as e:
    print(e)

```

A.1 Candidate Feature Identification

```

In [9]: feature_candidates = pd.DataFrame([
    {"source_dataset": "product.csv",
      "candidate_features": "product_line, class, style, days_to_manufacture",
      "rationale": "Structural product characteristics – line, grade, style"},
    {"source_dataset": "sales_order_detail.csv",
      "candidate_features": "total_quantity_sold, num_orders",
      "rationale": "Sales activity per product – to be aggregated in Section 4.1"},
    {"source_dataset": "special_offer_product.csv + special_offer.csv",
      "candidate_features": "num_discount_offers, max_discount_pct, has_voluntary_discount",
      "rationale": "Discount exposure per product – to be aggregated in Section 4.1"},
    {"source_dataset": "product_cost_history.csv",
      "candidate_features": "latest_standard_cost, cost_range",
      "rationale": "Cost level and cost stability per product"},
    {"source_dataset": "product_list_price_history.csv",
      "candidate_features": "latest_list_price, price_range",
      "rationale": "Price level and price stability per product"}
])

feature_candidates

```

	source_dataset	candidate_features	rationale
0	product.csv	product_line, class, style, days_to_manufacture	Structural product characteristics – line, grade, style
1	sales_order_detail.csv	total_quantity_sold, num_orders	Sales activity per product – to be aggregated in Section 4.1
2	special_offer_product.csv + special_offer.csv	num_discount_offers, max_discount_pct, has_voluntary_discount	Discount exposure per product – to be aggregated in Section 4.1
3	product_cost_history.csv	latest_standard_cost, cost_range	Cost level and cost stability per product
4	product_list_price_history.csv	latest_list_price, price_range	Price level and price stability per product

```

In [10]: feature_selection_1_insights = """
Six datasets across five feature groups are selected because they are the most relevant.
The remaining datasets are excluded for one of two reasons. sales_territory is not listed as a standalone source because it has no product-level features.
special_offer is not listed as a standalone source because it has no product-level features.
"""

```

```
In [11]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_1_insights', value=feature_s
feature_selection_1_insights
```

Six datasets across five feature groups are selected because they are the only available data that describes product-level commercial behaviour across distinct dimensions: what a product is structurally, how it performs in sales, what discount pressure it faces, what it costs to produce, and what price it commands. Together these dimensions give the clustering model evidence across multiple aspects of each product's commercial profile. The remaining datasets are excluded for one of two reasons. `sales_territory` has no `product_id` and cannot be directly joined to products. `customer`, `person`, `store`, `unit_measure`, `product_category`, and `product_sub_category` either have no `product_id` or describe entities and lookup classifications rather than product-level commercial characteristics. `special_offer` is not listed as a standalone source because it has no `product_id` — its discount data reaches products only through the `special_offer_product` bridge table, which is listed as the source for promotion features.

A.2 Join Coverage Check

```
In [12]: # Baseline: 254 sales-active products from sales_order_detail
sales_active_ids = set(sales_order_detail_df["product_id"].unique())
n = len(sales_active_ids)

cost_covered = product_cost_history_df[
    product_cost_history_df["product_id"].isin(sales_active_ids)
]["product_id"].nunique()

price_covered = product_list_price_history_df[
    product_list_price_history_df["product_id"].isin(sales_active_ids) &
    product_list_price_history_df["list_price"].notna()
]["product_id"].nunique()

promo_covered = special_offer_product_df[
    special_offer_product_df["product_id"].isin(sales_active_ids)
]["product_id"].nunique()

product_covered = product_df[
    product_df["product_id"].isin(sales_active_ids)
]["product_id"].nunique()

coverage_df = pd.DataFrame([
    {"source_dataset": "product.csv", "products_covered": product_covered},
    {"source_dataset": "product_cost_history.csv", "products_covered": cost_covered},
    {"source_dataset": "product_list_price_history.csv", "products_covered": price_covered},
    {"source_dataset": "special_offer_product.csv", "products_covered": promo_covered}
])
```

```
coverage_df
```

```
Out [12]:
```

	source_dataset	products_covered	out_of	missing
0	product.csv	254	254	0
1	product_cost_history.csv	254	254	0
2	product_list_price_history.csv	221	254	33
3	special_offer_product.csv	254	254	0

```
In [13]: feature_selection_2_insights = """
All four feature sources can be joined to the 254 sales-active products v
sales_order_detail defines the 254-product baseline and is not checked ag
"""
```

```
In [14]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_2_insights', value=feature_s
feature_selection_2_insights
```

All four feature sources can be joined to the 254 sales-active products via `product_id`. Three sources have complete coverage. The only gap is `product_list_price_history`, where 33 of the 254 sales-active products have no valid list price record. These 33 products are retained in the modelling population — their missing price and derived margin values will be imputed in Section E. `sales_order_detail` defines the 254-product baseline and is not checked against itself. `special_offer` has no `product_id` column and connects to products only through the `special_offer_product` bridge table, so `special_offer_product` represents promotion coverage in the check.

A.3 Feature Selection from Source Data

```
In [15]: # Verify is_sellable is constant among sales-active products
# sales_active_ids defined in A.2
product_active = product_df[product_df["product_id"].isin(sales_active_id

print("is_sellable value distribution among 254 sales-active products:")
print(product_active["is_sellable"].value_counts())

# Features confirmed from source data – exist as raw or extractable colum
features_confirmed = [
    "product_line", "class", "style", "days_to_manufacture", "is_manufact
    "latest_standard_cost", "cost_range",
    "latest_list_price", "price_range",
]

print(f"\n{len(features_confirmed)} features confirmed from source data:")
for f in features_confirmed:
    print(f" {f}")
```

```
is_sellable value distribution among 254 sales-active products:
is_sellable
1.0      254
Name: count, dtype: int64
```

9 features confirmed from source data:

```
product_line
class
style
days_to_manufacture
is_manufactured
latest_standard_cost
cost_range
latest_list_price
price_range
```

```
In [16]: feature_selection_explanations = """
9 features are selected from source data. is_sellable is excluded — it is
From product_cost_history.csv, latest_standard_cost captures each product
"""
```

```
In [17]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_explanations', value=feature
```

feature_selection_explanations

9 features are selected from source data. `is_sellable` is excluded — it is 1.0 for every one of the 254 sales-active products, meaning every product in the clustering population has the same value. A constant feature provides no basis for distinguishing between products and contributes nothing to cluster discovery. The remaining five `product.csv` candidates all vary across products and are retained. From `product_cost_history.csv`, `latest_standard_cost` captures each product's current cost position and `cost_range` captures cost stability over time. From `product_list_price_history.csv`, `latest_list_price` captures current price position and `price_range` captures price stability.

B. Data Cleaning

B.1 Fixing Duplicate Rows in `product.csv`

```
In [18]: # Show the issue
print(f"product_df shape before dedup: {product_df.shape}")
print(f"Unique product_ids: {product_df['product_id'].nunique()}")
print(f"Fully duplicate rows: {product_df.duplicated().sum()}")

# Fix: drop fully duplicate rows, keep first occurrence
product_df = product_df.drop_duplicates(subset=["product_id"]).reset_index

# Verify
```

```
print(f"\nproduct_df shape after dedup: {product_df.shape}")
print(f"Unique product_ids after dedup: {product_df['product_id'].nunique}")
```

```
product_df shape before dedup: (886, 23)
Unique product_ids: 504
Fully duplicate rows: 382
```

```
product_df shape after dedup: (504, 23)
Unique product_ids after dedup: 504
```

```
In [19]: data_cleaning_1_explanations = """
product.csv contains 886 rows but only 504 unique product_ids. The 382 ex

Leaving duplicates in place would inflate join results when product.csv i
"""
```

```
In [20]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_clea
```

data_cleaning_1_explanations

product.csv contains 886 rows but only 504 unique product_ids. The 382 extra rows are exact duplicates — every column value is identical to another row for the same product. This is a data export artefact, not intentional versioning. Leaving duplicates in place would inflate join results when product.csv is merged with other datasets: one product_id would match multiple rows, producing duplicate records in the assembled modelling table. Dropping duplicates on product_id reduces the table to 504 rows, one per product, which is the correct grain for product-level clustering.

B.2 Fixing Invalid and Duplicate Rows in product_list_price_history.csv

```
In [21]: # Show the issues
print(f"product_list_price_history_df shape: {product_list_price_history_
print(f"Rows with null list_price: {product_list_price_history_df['list_p
print(f"Fully duplicate rows: {product_list_price_history_df.duplicated()

# Fix 1: drop rows with no list_price – a price record with no value is u
product_list_price_history_df = product_list_price_history_df[
    product_list_price_history_df['list_price'].notna()
].reset_index(drop=True)

# Fix 2: drop fully duplicate rows
product_list_price_history_df = product_list_price_history_df.drop_duplic

# Verify
print(f"\nproduct_list_price_history_df shape after cleaning: {product_li
print(f"Unique product_ids: {product_list_price_history_df['product_id'].
```

```
product_list_price_history_df shape: (620, 4)
Rows with null list_price: 145
Fully duplicate rows: 225
```

```
product_list_price_history_df shape after cleaning: (328, 4)
Unique product_ids: 253
```

```
In [22]: data_cleaning_2_explanations = """
product_list_price_history.csv has two issues that need to be resolved before the table can be used to extract price features.

The first issue is 145 rows where list_price is null. A price history record with no price value is meaningless — it cannot contribute to latest price or price range calculations. These rows are dropped.

The second issue is 225 fully duplicate rows, identical across all four columns. As with product.csv, this is a data export artefact. Duplicates are dropped after the null removal step, leaving 328 rows across 253 unique products.

19 rows in the cleaned table have a null start_date but a valid list_price. These are retained — the price value is usable for price range calculation. The null start_date is handled during feature assembly in Section C when selecting the latest price record per product.
"""
```

```
In [23]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_2_explanations', value=data_cleaning_2_explanations)
```

data_cleaning_2_explanations

product_list_price_history.csv has two issues that need to be resolved before the table can be used to extract price features. The first issue is 145 rows where list_price is null. A price history record with no price value is meaningless — it cannot contribute to latest price or price range calculations. These rows are dropped. The second issue is 225 fully duplicate rows, identical across all four columns. As with product.csv, this is a data export artefact. Duplicates are dropped after the null removal step, leaving 328 rows across 253 unique products. 19 rows in the cleaned table have a null start_date but a valid list_price. These are retained — the price value is usable for price range calculation. The null start_date is handled during feature assembly in Section C when selecting the latest price record per product.

C. Split Datasets

```
In [24]: # sales_active_ids defined in A.2 – 254 sales-active products

# Product attributes from product.csv (already deduplicated in B.1)
product_features = product_df[product_df["product_id"].isin(sales_active_
    "product_id", "product_line", "class", "style", "days_to_manufacture"
)]

# Latest cost and cost stability from product_cost_history.csv
cost_features = product_cost_history_df.sort_values("start_date").groupby(
    latest_standard_cost=("standard_cost", "last"),
    cost_range=("standard_cost", lambda x: round(x.max() - x.min(), 4))
).reset_index()

# Latest valid price and price stability from product_list_price_history.
```



```

# null start_date rows sorted first so the most recent dated record is se
price_features = product_list_price_history_df.sort_values(
    "start_date", na_position="first"
).groupby("product_id").agg(
    latest_list_price=("list_price", "last"),
    price_range=("list_price", lambda x: round(x.max() - x.min(), 4))
).reset_index()

# Assemble 254-product base table
training_df = pd.DataFrame({"product_id": sorted(sales_active_ids)})
training_df = training_df.merge(product_features, on="product_id", how="l
training_df = training_df.merge(cost_features, on="product_id", how="l
training_df = training_df.merge(price_features, on="product_id", how="l

# No train/val/test split – unsupervised task uses the full population
validation_df = pd.DataFrame(columns=training_df.columns)
testing_df = pd.DataFrame(columns=training_df.columns)

print(f"training_df shape: {training_df.shape}")
print(f"Columns: {list(training_df.columns)}")
print(f"\nMissing values:")
print(training_df.isnull().sum()[training_df.isnull().sum() > 0])

```

training_df shape: (254, 10)

Columns: ['product_id', 'product_line', 'class', 'style', 'days_to_manufac
ture', 'is_manufactured', 'latest_standard_cost', 'cost_range', 'latest_li
st_price', 'price_range']

Missing values:

product_line	15
class	56
style	72
latest_list_price	33
price_range	33
dtype:	int64

In [25]: data_splitting_explanations = """
Clustering is an unsupervised task – there is no target variable and no l

All 254 sales-active products are used as the modelling population. Withh

validation_df and testing_df are set to empty DataFrames. No split is per
"""

In [26]: # Do not modify this code
print_tile(size="h3", key='data_splitting_explanations', value=data_split

Clustering is an unsupervised task — there is no target variable and no labelled ground truth to evaluate predictions against. Held-out validation and test sets exist to measure how well a supervised model generalises to unseen data, which is not a relevant concern for clustering. All 254 sales-active products are used as the modelling population. Withholding products would reduce the dataset without any benefit — there are no labels to protect from leakage and no generalisation performance to measure. Cluster quality is assessed using internal metrics such as silhouette score and inertia, which operate on the same data used for fitting. `validation_df` and `testing_df` are set to empty DataFrames. No split is performed.

D. Feature Engineering

```
In [27]: # DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
    training_df_eng = training_df.copy()
    validation_df_eng = validation_df.copy()
    testing_df_eng = testing_df.copy()
except Exception as e:
    print(e)
```

D.1 Sales Behaviour Features

```
In [28]: # Aggregate sales activity per product from transaction-level records
sales_agg = sales_order_detail_df.groupby("product_id").agg(
    total_quantity_sold=("order_quantity", "sum"),
    num_orders=("sales_order_id", "nunique") # distinct orders, not rows
).reset_index()

# Left join preserves all 254 products in population order
training_df_eng = training_df_eng.merge(sales_agg, on="product_id", how="left")

print(f"Shape after D.1: {training_df_eng.shape}")
print(f"\nNew columns – null check:")
print(training_df_eng[["total_quantity_sold", "num_orders"]].isnull().sum)
print(f"\nDistribution:")
print(training_df_eng[["total_quantity_sold", "num_orders"]].describe().r
```

Shape after D.1: (254, 12)

New columns – null check:

```
total_quantity_sold    0
num_orders              0
dtype: int64
```

Distribution:

	total_quantity_sold	num_orders
count	254.00	254.00
mean	281.26	165.75
std	407.73	287.08
min	1.00	1.00
25%	67.25	30.00
50%	132.50	53.50
75%	315.75	173.25
max	2837.00	2156.00

```
In [29]: feature_engineering_1_explanations = """
total_quantity_sold and num_orders are aggregated from sales_order_detail

total_quantity_sold captures the total units sold per product across all

num_orders counts the distinct sales orders containing each product. A pr

Both features have no missing values – all 254 sales-active products have
"""
```

```
In [30]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_1_explanations', value=fea
```

feature_engineering_1_explanations

total_quantity_sold and **num_orders** are aggregated from **sales_order_detail** at the product level. **total_quantity_sold** captures the total units sold per product across all recorded transactions. Products with high total volume generate proportionally more revenue exposure — their margin position has greater financial impact on the business. **num_orders** counts the distinct sales orders containing each product. A product with high **num_orders** is consistently and repeatedly demanded across many transactions, while a product with low **num_orders** appears infrequently. This distinction is relevant to margin risk because consistent demand reduces the need for promotional discounting to clear stock. Both features have no missing values — all 254 sales-active products have at least one transaction record by definition.

D.2 Promotion Exposure Features

```
In [31]: # Join offer type and discount rate onto the bridge table
offers = special_offer_product_df.merge(
    special_offer_df[["special_offer_id", "type", "discount_pct"]],
    on="special_offer_id", how="left"
)
```

```

# Exclude "No Discount" – discount_pct=0, linked to all 295 bridge-table
# including it would give every product an offer count regardless of real
real_offers = offers[offers["type"] != "No Discount"]

# Aggregate real discount exposure per product
promo_agg = real_offers.groupby("product_id").agg(
    num_discount_offers=("special_offer_id", "nunique"),
    max_discount_pct=("discount_pct", "max"),
    has_volume_discount=("type", lambda x: int("Volume Discount" in x.val
)).reset_index()

# Left join – products with no real offers receive NaN, filled with 0
training_df_eng = training_df_eng.merge(promo_agg, on="product_id", how="left")
training_df_eng[["num_discount_offers", "max_discount_pct", "has_volume_discount"]] = training_df_eng[["num_discount_offers", "max_discount_pct", "has_volume_discount"].fillna(0)

print(f"Shape after D.2: {training_df_eng.shape}")
print(f"\nProducts with no real discount offers: {(training_df_eng['num_discount_offers'] == 0).sum()}")
print(f"Products with at least one real offer: {(training_df_eng['num_discount_offers'] > 0).sum()}")
print(f"\nDistribution:")
print(training_df_eng[["num_discount_offers", "max_discount_pct", "has_volume_discount"]].describe())

```

Shape after D.2: (254, 15)

Products with no real discount offers: 112

Products with at least one real offer: 142

Distribution:

	num_discount_offers	max_discount_pct	has_volume_discount
count	254.0000	254.0000	254.0000
mean	0.9291	0.0822	0.4331
std	1.0306	0.1378	0.4965
min	0.0000	0.0000	0.0000
25%	0.0000	0.0000	0.0000
50%	1.0000	0.0200	0.0000
75%	2.0000	0.1000	1.0000
max	4.0000	0.5000	1.0000

In [32]: feature_engineering_2_explanations = """
Three promotion exposure features are aggregated from special_offer_produ

"No Discount" offers are excluded before aggregation. As identified in SD

num_discount_offers counts distinct real discount offers linked to each p

112 of 254 products have no real discount offer and receive 0 for all thr
"""

In [33]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_2_explanations', value=fea

Three promotion exposure features are aggregated from special_offer_product joined with special_offer. "No Discount" offers are excluded before aggregation. As identified in SD-3 EDA, this offer type has discount_pct = 0 and is linked to all 295 bridge-table products. Including it would give every product at least one offer count regardless of whether it ever received a real discount, making the feature uninformative. num_discount_offers counts distinct real discount offers linked to each product. max_discount_pct captures the highest discount rate the product has been exposed to across all its offers. has_volume_discount flags products linked to volume discount offers — a specific offer type where discount is contingent on purchasing above a minimum quantity threshold, indicating price sensitivity to order size. 112 of 254 products have no real discount offer and receive 0 for all three features. 142 products have at least one real offer.

D.3 Margin Features

```
In [34]: # Derive margin from cost and price columns assembled in C
training_df_eng["estimated_margin"] = (
    training_df_eng["latest_list_price"] - training_df_eng["latest_standa
)
training_df_eng["estimated_margin_pct"] = (
    training_df_eng["estimated_margin"] / training_df_eng["latest_list_pr
)

print(f"Shape after D.3: {training_df_eng.shape}")
print(f"\nNew columns - null check:")
print(training_df_eng[["estimated_margin", "estimated_margin_pct"]].isnul
print(f"\nDistribution:")
print(training_df_eng[["estimated_margin", "estimated_margin_pct"]].descr
```

Shape after D.3: (254, 17)

New columns - null check:

```
estimated_margin      33
estimated_margin_pct  33
dtype: int64
```

Distribution:

	estimated_margin	estimated_margin_pct
count	221.0000	221.0000
mean	306.9470	0.4637
std	367.5217	0.1035
min	1.4335	0.2300
25%	39.7510	0.3784
50%	162.9407	0.4524
75%	402.1663	0.5560
max	1487.8356	0.6260

```
In [35]: feature_engineering_3_explanations = """
estimated_margin and estimated_margin_pct are derived from the cost and p
```

estimated_margin is the absolute difference between latest_list_price and what the product earns and what it costs to produce. estimated_margin_pct list price, making products with different price scales comparable on a n

Both features are NaN for the 33 products that have no list price history population median in Section E, alongside the price columns on which they

```
In [36]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_3_explanations', value=fea
```

feature_engineering_3_explanations

estimated_margin and estimated_margin_pct are derived from the cost and price columns already in training_df_eng. estimated_margin is the absolute difference between latest_list_price and latest_standard_cost — the gap between what the product earns and what it costs to produce. estimated_margin_pct expresses that gap as a proportion of list price, making products with different price scales comparable on a normalised basis. Both features are NaN for the 33 products that have no list price history. These will be imputed with the population median in Section E, alongside the price columns on which they depend.

D.4 Final Feature Confirmation

```
In [37]: # Confirm all engineered features are present in training_df_eng
feature_cols = [c for c in training_df_eng.columns if c != "product_id"]

print(f"training_df_eng shape: {training_df_eng.shape}")
print(f"\nAll feature columns ({len(feature_cols)}):")
for f in feature_cols:
    print(f"    {f}")

null_counts = training_df_eng[feature_cols].isnull().sum()
print(f"\nNull counts (non-zero only):")
print(null_counts[null_counts > 0])
```

```
training_df_eng shape: (254, 17)
```

```
All feature columns (16):
```

```
product_line
class
style
days_to_manufacture
is_manufactured
latest_standard_cost
cost_range
latest_list_price
price_range
total_quantity_sold
num_orders
num_discount_offers
max_discount_pct
has_volume_discount
estimated_margin
estimated_margin_pct
```

```
Null counts (non-zero only):
```

```
product_line      15
class             56
style             72
latest_list_price 33
price_range       33
estimated_margin  33
estimated_margin_pct 33
dtype: int64
```

```
In [38]: feature_engineering_n_explanations = """
16 features are now present in training_df_eng: 9 carried from source data
(price history), 2 aggregated from sales transactions (D.1), 3 aggregated
and 2 derived from cost and price (D.3).

training_df_eng shape is (254, 17) – 254 products and 16 feature columns

Null values remain in product_line (15 products), class (56 products), style
and price_range (33 products), and estimated_margin and estimated_margin_pct
(price). These are resolved in Section E before modelling.
"""
```

```
In [39]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_n_explanations', value=fea
```

16 features are now present in `training_df_eng`: 9 carried from source data (product attributes, cost history, price history), 2 aggregated from sales transactions (D.1), 3 aggregated from discount offer data (D.2), and 2 derived from cost and price (D.3). `training_df_eng` shape is (254, 17) — 254 products and 16 feature columns plus `product_id`. Null values remain in `product_line` (15 products), `class` (56 products), `style` (72 products), `latest_list_price` and `price_range` (33 products), and `estimated_margin` and `estimated_margin_pct` (33 products, derived from list price). These are resolved in Section E before modelling.

E. Data Preparation for Modeling

In [40]: *# DO NOT MODIFY THE CODE IN THIS CELL*
Create copy of datasets

```
try:
    X_train = training_df_eng.copy()
    X_val = validation_df_eng.copy()
    X_test = testing_df_eng.copy()
except Exception as e:
    print(e)
```

E.1 Imputation

In [41]: *# Nominal categoricals: fill with 'Unknown' — becomes a distinct one-hot*
`X_train["product_line"] = X_train["product_line"].fillna("Unknown")`
`X_train["style"] = X_train["style"].fillna("Unknown")`

Ordinal categorical: mode preserves the L/M/H scale for ordinal encoding
`class_mode = X_train["class"].mode()[0]`
`X_train["class"] = X_train["class"].fillna(class_mode)`

Numeric: population median is robust to right-skewed price and margin d
`numeric_cols = ["latest_list_price", "price_range", "estimated_margin", "`
`medians = {}`
`for col in numeric_cols:`
 `medians[col] = X_train[col].median()`
 `X_train[col] = X_train[col].fillna(medians[col])`

`print(f"class mode used for imputation: {class_mode}")`
`print(f"\nMedian values used for numeric imputation:")`
`for col, val in medians.items():`
 `print(f" {col}: {val:.4f}")`
`print(f"\nNull count after imputation: {X_train.isnull().sum().sum()}")`


```
class mode used for imputation: L
```

```
Median values used for numeric imputation:
```

```
latest_list_price: 337.2200  
price_range: 0.0000  
estimated_margin: 162.9407  
estimated_margin_pct: 0.4524
```

```
Null count after imputation: 0
```

```
In [42]: data_transformation_1_explanations = """  
Missing values in seven columns are resolved before encoding and scaling.  
  
product_line (15 products) and style (72 products) are filled with 'Unkno  
that will be one-hot encoded in E.2 – 'Unknown' becomes a distinct binary  
which known category the missing products belong to. The style group is l  
that mode imputation would introduce substantial false signal.  
  
class (56 products) is filled with its mode – the most common class value  
class. class has a natural L < M < H ordering that ordinal encoding prese  
meaningful position on that scale, so mode is the only imputation that ke  
  
latest_list_price, price_range, estimated_margin, and estimated_margin_pc  
products with no list price history) are filled with their population med  
mean because price and margin distributions are typically right-skewed; t  
high-value outliers.  
"""
```

```
In [43]: # DO NOT MODIFY THE CODE IN THIS CELL  
print_tile(size="h3", key='data_transformation_1_explanations', value=dat
```

```
data_transformation_1_explanations
```

Missing values in seven columns are resolved before encoding and scaling. product_line (15 products) and style (72 products) are filled with 'Unknown'. Both are nominal variables that will be one-hot encoded in E.2 — 'Unknown' becomes a distinct binary column rather than asserting which known category the missing products belong to. The style group is large enough (28% of products) that mode imputation would introduce substantial false signal. class (56 products) is filled with its mode — the most common class value among products with a known class. class has a natural L M H ordering that ordinal encoding preserves. 'Unknown' has no meaningful position on that scale, so mode is the only imputation that keeps the ordinal encoding valid. latest_list_price, price_range, estimated_margin, and estimated_margin_pct (all 33 products — the same products with no list price history) are filled with their population medians. Median is preferred over mean because price and margin distributions are typically right-skewed; the median is not inflated by high-value outliers.

E.2 Encoding

```
In [44]: # Ordinal encode class – preserves  $L < M < H$  ordering for Euclidean distance
class_map = {"L": 0, "M": 1, "H": 2}
X_train["class"] = X_train["class"].map(class_map)

# One-hot encode nominal categorical – drop_first=False retains all categories
# For clustering with Euclidean distance, dropping a reference category can distort
X_train = pd.get_dummies(X_train, columns=["product_line", "style"], drop

print(f"Shape after encoding: {X_train.shape}")
print(f"\nclass encoding (value counts):")
print(X_train["class"].value_counts().sort_index())
print(f"\nOne-hot columns created:")
ohe_cols = [c for c in X_train.columns if c.startswith("product_line_") or
for c in ohe_cols:
    print(f"    {c}: {X_train[c].sum()} products")
```

Shape after encoding: (254, 24)

class encoding (value counts):

```
class
0      137
1       46
2       71
```

Name: count, dtype: int64

One-hot columns created:

```
product_line_M: 83 products
product_line_R: 75 products
product_line_S: 34 products
product_line_T: 47 products
product_line_Unknown: 15 products
style_M: 6 products
style_U: 149 products
style_Unknown: 72 products
style_W: 27 products
```

In [44]:

```
In [45]: data_transformation_2_explanations = """
Two encoding strategies are applied based on the measurement scale of each
class is ordinal encoded: L maps to 0, M to 1, H to 2. This preserves the
variable – the distance between L and H ( $|2-0|=2$ ) is twice the distance between
M and L ( $|1-0|=1$ ), which reflects the three-tier structure in Euclidean distance
three clustering algorithms.

product_line and style are one-hot encoded. Both are nominal – there is no
product lines or style codes, so each category is treated as equidistant.
all categories including 'Unknown', which represents a distinct group of
classification assigned. Dropping a reference category in clustering creates
distances between categories and is not appropriate here.
"""
```

```
In [46]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_2_explanations', value=data_transformation_2_explanations)
```

Two encoding strategies are applied based on the measurement scale of each variable. class is ordinal encoded: L maps to 0, M to 1, H to 2. This preserves the natural ordering of the variable — the distance between L and H ($|2-0|=2$) is twice the distance between adjacent classes ($|1-0|=1$), which reflects the three-tier structure in Euclidean distance calculations used by all three clustering algorithms. product_line and style are one-hot encoded. Both are nominal — there is no natural ordering between product lines or style codes, so each category is treated as equidistant. drop_first=False retains all categories including 'Unknown', which represents a distinct group of products with no classification assigned. Dropping a reference category in clustering creates asymmetric Euclidean distances between categories and is not appropriate here.

E.3 Scaling

```
In [47]: from sklearn.preprocessing import StandardScaler

# Binary columns are already on [0,1] scale – scaling would distort their
binary_cols = (
    ["is_manufactured", "has_volume_discount"] +
    [c for c in X_train.columns if c.startswith("product_line_") or c.sta
)

# Scale all remaining numeric columns (continuous + ordinal)
cols_to_scale = [c for c in X_train.columns if c != "product_id" and c no

scaler = StandardScaler()
X_train[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])

print(f"Columns scaled ({len(cols_to_scale)}):")
for c in cols_to_scale:
    print(f"  {c}")
print(f"\nBinary columns left unchanged ({len(binary_cols)}):")
for c in binary_cols:
    print(f"  {c}")
print(f"\nX_train shape: {X_train.shape}")
```

```
Columns scaled (12):
class
days_to_manufacture
latest_standard_cost
cost_range
latest_list_price
price_range
total_quantity_sold
num_orders
num_discount_offers
max_discount_pct
estimated_margin
estimated_margin_pct
```

```
Binary columns left unchanged (11):
is_manufactured
has_volume_discount
product_line_M
product_line_R
product_line_S
product_line_T
product_line_Unknown
style_M
style_U
style_Unknown
style_W
```

```
X_train shape: (254, 24)
```

```
In [47]:
```

```
In [48]: data_transformation_3_explanations = """
StandardScaler is applied to all continuous and ordinal numeric columns.
feature to mean=0 and standard deviation=1, ensuring that features with l
such as latest_list_price or total_quantity_sold – do not dominate Euclid
in KMeans, Mean Shift, and Agglomerative clustering.

Binary columns – is_manufactured, has_volume_discount, and all one-hot en
style columns – are left unchanged. These features are already on a [0,1]
StandardScaler to binary columns centres them around their mean and produ
distorting the categorical signal they represent.
"""
```

```
In [49]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_3_explanations', value=dat
```

StandardScaler is applied to all continuous and ordinal numeric columns. Scaling transforms each feature to mean=0 and standard deviation=1, ensuring that features with large absolute ranges — such as latest_list_price or total_quantity_sold — do not dominate Euclidean distance calculations in KMeans, Mean Shift, and Agglomerative clustering. Binary columns — is_manufactured, has_volume_discount, and all one-hot encoded product_line and style columns — are left unchanged. These features are already on a [0,1] scale. Applying StandardScaler to binary columns centres them around their mean and produces non-binary values, distorting the categorical signal they represent.

E.4 Variance and Correlation Check

```
In [50]: feature_cols = [c for c in X_train.columns if c != "product_id"]

# --- Variance check ---
# Scaled continuous features have variance=1 by construction; check focus
binary_check_cols = [
    c for c in feature_cols
    if c.startswith("product_line_") or c.startswith("style_")
    or c in ["is_manufactured", "has_volume_discount"]
]
variances = X_train[binary_check_cols].var().sort_values()
var_threshold = 0.05 #  $p*(1-p) < 0.05$  means  $< 5.5\%$  or  $> 94.5\%$  prevalence
low_var_cols = variances[variances < var_threshold].index.tolist()

print("=== Variance - Binary/One-hot Columns ===")
print(variances.round(4).to_string())
print(f"\nDropped (variance < {var_threshold}): {low_var_cols} if low_var_")
if low_var_cols:
    X_train = X_train.drop(columns=low_var_cols)

# --- Correlation check ---
feature_cols = [c for c in X_train.columns if c != "product_id"]
corr_matrix = X_train[feature_cols].corr().abs()
corr_threshold = 0.85 #  $r > 0.85$  means features share  $> 72\%$  of their var

high_corr = [
    (corr_matrix.columns[i], corr_matrix.columns[j], round(corr_matrix.ilo
    for i in range(len(corr_matrix.columns))
    for j in range(i + 1, len(corr_matrix.columns))
    if corr_matrix.iloc[i, j] > corr_threshold
]

print("\n=== Correlation - All Features ===")
print(f"Pairs with  $|r| > {corr\_threshold}$ :")
if high_corr:
    for a, b, r in sorted(high_corr, key=lambda x: -x[2]):
        print(f" {a} - {b}:  $r={r:.4f}$ ")
else:
    print(" None")
```

```
# --- Domain-driven removal decisions ---
# estimated_margin: r=0.994 with latest_list_price - near-linear shift of
# estimated_margin_pct is retained as it normalises for price scale and c
# num_orders: r=0.913 with total_quantity_sold - volume is the more direc
# latest_standard_cost and latest_list_price kept despite r=0.900 - diffe
# cost_range and price_range kept despite r=0.864 - different volatility
corr_drop_cols = ["estimated_margin", "num_orders"]
X_train = X_train.drop(columns=corr_drop_cols)

print(f"\nDropped (domain reasoning): {corr_drop_cols}")
print(f"\nX_train shape after E.4: {X_train.shape}")
print(f"Final feature count (excl. product_id): {X_train.shape[1] - 1}")
```

=== Variance - Binary/One-hot Columns ===

style_M	0.0232
product_line_Unknown	0.0558
style_W	0.0954
product_line_S	0.1164
product_line_T	0.1514
style_Unknown	0.2039
is_manufactured	0.2073
product_line_R	0.2089
product_line_M	0.2209
style_U	0.2435
has_volume_discount	0.2465

Dropped (variance < 0.05): ['style_M']

=== Correlation - All Features ===

Pairs with $|r| > 0.85$:

```
latest_list_price - estimated_margin: r=0.9940
total_quantity_sold - num_orders: r=0.9134
latest_standard_cost - latest_list_price: r=0.9008
latest_standard_cost - estimated_margin: r=0.8911
cost_range - price_range: r=0.8635
```

Dropped (domain reasoning): ['estimated_margin', 'num_orders']

X_train shape after E.4: (254, 21)

Final feature count (excl. product_id): 20

In [51]: data_transformation_n_explanations = """

Two checks are applied to the fully prepared feature table before passing

Variance check: binary and one-hot columns are assessed against a threshold. For a categorical feature, $\text{variance} = p \times (1-p)$, so this threshold corresponds to fewer than 5 products sharing the same value. A subgroup this small cannot anchor a model and therefore does not support a business decision. style_M (variance=0.023, ~2.4% prevalence) is dropped. Scaled continuous features are excluded from this check - StandardScaler handles them by construction.

Correlation check: all feature pairs are assessed for absolute Pearson correlation. Features with a correlation equivalent to features sharing more than 72% of their variance (r^2). The threshold is a heuristic, not a hard rule - it surfaces candidates for domain review rather than automatic removal. Five pairs are flagged.

estimated_margin is dropped: r=0.994 with latest_list_price. When standard cost is stable, margin becomes a near-linear shift of price - it adds no independent information and is retained as it normalises for price scale and captures margin efficiency.

num_orders is dropped: $r=0.913$ with total_quantity_sold. Products that appear to sell high volumes – the two features move together. total_quantity_sold is a more direct measure of financial exposure.

latest_standard_cost and latest_list_price are retained despite $r=0.900$. Higher-cost products tend to be priced higher, but the two features represent fundamentally different business dimensions – production cost and market price. Dropping either loses the absolute scale signal for that dimension.

cost_range and price_range are retained despite $r=0.864$. Both capture historical volatility but in different commercial dimensions – production cost history and list price history respectively.

```
In [52]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_n_explanations', value=data_transformation_n_explanations)
```

data_transformation_n_explanations

Two checks are applied to the fully prepared feature table before passing it to Section F. Variance check: binary and one-hot columns are assessed against a threshold of 0.05. For a binary feature, $\text{variance} = p(1-p)$, so this threshold corresponds to fewer than 5.5% or more than 94.5% of products sharing the same value. A subgroup this small cannot anchor a meaningful cluster or support a business decision. style_M (variance=0.023, ~2.4% prevalence) is dropped on this basis. Scaled continuous features are excluded from this check – StandardScaler sets their variance to 1 by construction. Correlation check: all feature pairs are assessed for absolute Pearson correlation above 0.85, equivalent to features sharing more than 72% of their variance (r^2). The threshold is a screening device, not a hard rule – it surfaces candidates for domain review rather than triggering automatic removal. Five pairs are flagged. estimated_margin is dropped: $r=0.994$ with latest_list_price. When standard_cost is relatively stable, margin becomes a near-linear shift of price – it adds no independent signal. estimated_margin_pct is retained as it normalises for price scale and captures margin efficiency independently of magnitude. num_orders is dropped: $r=0.913$ with total_quantity_sold. Products that appear in many orders also tend to sell high volumes – the two features move together. total_quantity_sold is retained as the more direct measure of financial exposure. latest_standard_cost and latest_list_price are retained despite $r=0.900$. Higher-cost products tend to be priced higher, but the two features represent fundamentally different business dimensions – production cost and market price. Dropping either loses the absolute scale signal for that dimension. cost_range and price_range are retained despite $r=0.864$. Both capture historical volatility but in different commercial dimensions – production cost history and list price history respectively.

```
In [53]: # Clustering is unsupervised – no target labels exist.  
# y files are saved as empty DataFrames so downstream notebooks  
# that load y_train.csv do not fail with a parse error.  
y_train = pd.DataFrame()  
y_val = pd.DataFrame()  
y_test = pd.DataFrame()  
  
y_train.to_csv(at.folder_path / "y_train.csv", index=False)  
y_val.to_csv(at.folder_path / "y_val.csv", index=False)  
y_test.to_csv(at.folder_path / "y_test.csv", index=False)  
  
print("y_train.csv, y_val.csv, y_test.csv saved (empty – unsupervised tas  
y_train.csv, y_val.csv, y_test.csv saved (empty – unsupervised task)
```

F. Save Datasets

Do not change this code

```
In [54]: # DO NOT MODIFY THE CODE IN THIS CELL  
  
try:  
    X_train.to_csv(at.folder_path / 'X_train.csv', index=False)  
  
    X_val.to_csv(at.folder_path / 'X_val.csv', index=False)  
  
    X_test.to_csv(at.folder_path / 'X_test.csv', index=False)  
except Exception as e:  
    print(e)
```